3️⃣

MATERI WEEK 03 - Thread

A. REVIEW SESI LECTURE

1. Pendahuluan

Pada zaman sekarang, beberapa aplikasi modern menggunakan konsep Multi-Threaded. Thread berjalan didalam aplikasi tersebut. Beberap[a tugas dalam aplikasi tersebut dapat diimplementasikan sebagai thread-thread yang terpisah contohnya:

- Update Display
- Fetch Data
- Spell Checking
- Answer a Network Request, dll

Thread Creation merupakan Light-Weigth sedangkan Process Creation adalah Heavy-Weigth. Tentunya thread dapat menyederhakan sebuah kode dan meningkatkan efisiensinya.

- Kernel secara umum juga menggunakan konsep multithreaded

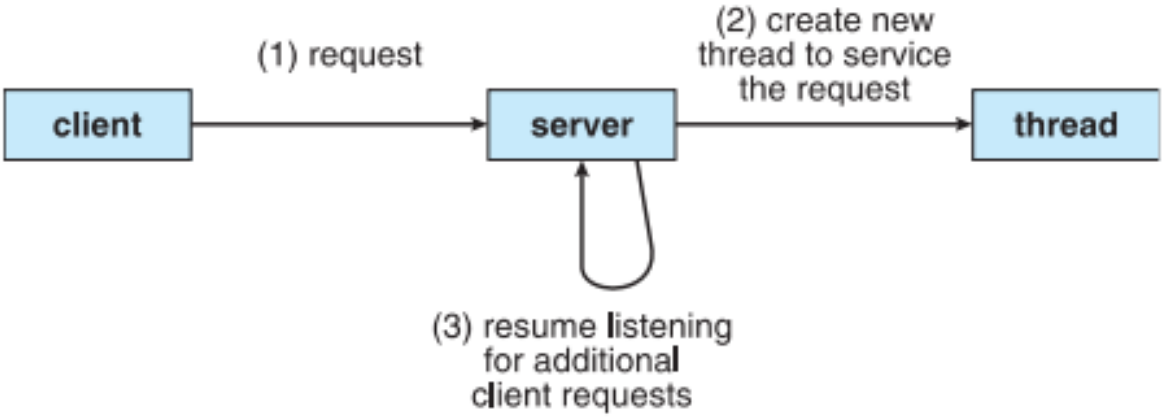
2. Thread

Pada umumnya sebuah Process memiliki resourcenya sendiri ddan memiliki sebuah Thread of Execution. Tapi tidak menutup kemungkinan bahwa dalam 1 Process memiliki lebih dari 1 thread.

Konsep inilah yang kita sebut sebagai multithreading. Sebagai ilustrasi, bayangkan dalam sebuah Process, kita memiliki lebih dari 1 program counter sehingga kita dapat mengeksekusi lebih dari 1 kode yang berbeda lokasinya pada 1 waktu yang sama.

Jadi, Thread adalah sebuah aliran / alur program, dalam sebuah process pasti setidaknya memiliki 1 thread atau lebih.

Contoh Thread dalam Server Architecture :



Thread memiliki beberapa kelebihan yaitu :

- **Responsiveness** → Dapat memungkinkan eksekusi tetap berlanjut jika sebagian dari process terblokir, pentin untuk User Interfaces
- **Resource Sharing** → Masing-masing thread dapat membagikan resource yang digunakan oleh Process, lebih mudah dibandingkan Shared Memory dan Message Passing yang ada di IPC Process.
- **Economy** → Lebih murah dibanding Process Creation, Thread dapat melakukan switch dengan beban yang lebih rendan dibanding context switch Process.
- **Scalability** → Process dapat mengambil keuntungan dari MultiProcessor Architectures.

3. MultiCore Programming

Multicore atau Multiprocessor System memberikan tantangan baru kepada Programmer yaitu :

- Dividing Activities

- Balance
- Data Splitting
- Data Dependency
- Testing and Debugging

Konsep Paralelisme yang diterapkan pada sistem memungkinkan sistem dapat melakukan lebih dari 1 tugas secara simultan / bersamaan.

Concurrency mendukung lebih dari 1 tugas dijalankan, tetapi hal ini juga berlaku pada processor dengan 1 core yang memberikan Concurrency lewat Scheduler.

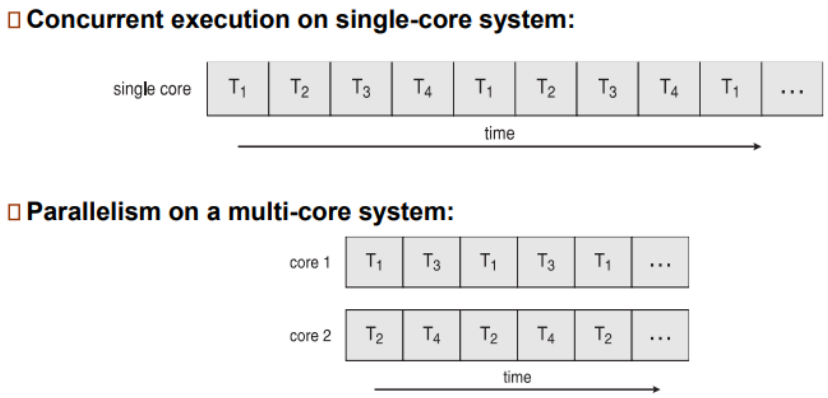
Beberapa Tipe Paralelisme :

- Data Paralelism → Mendistribusikan bagian dari data yang sama keseluruh core, dengan operasi yang sama pada setiap core.
- Task Paralelism → Mendistribusikan Threads keseluruh core, setiap thread akan menjalankan operasi yang berbeda-beda.

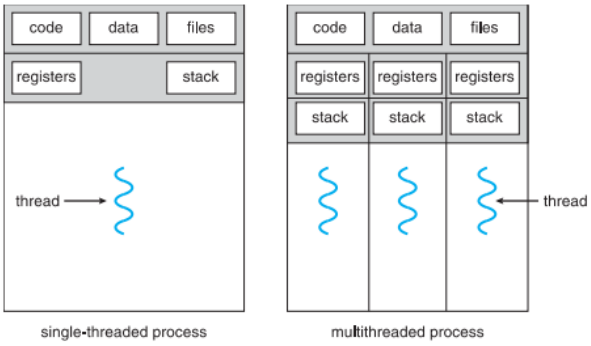
Semakin Konsep Threads berkembang, Bagaimana Arsitektur Hardware dapat mendukung Threading ?

1. CPUs memiliki core yang mirip dengan hardware threads.
2. Contohnya seperti Oracle SPARCT4 dengan 8 Cores dan 8 Hardware Threads per Core.

Berikut Perbedaaan Concurrency vs Parallelism :



Single and MultiThreaded Process :



4. Amdahl's Law

Hukum Amdahl mengidentifikasi peningkatan kinerja dengan penambahan Core pada CPU dalam sebuah aplikasi baik yang memiliki kompoen serial dan paralel.

- S adalah bagian yang bersifat serial
- N adalah Banyak Core

Berarti, jika sebuah aplikasi memiliki 75% Paralel / 25% Serial, bergerak dari 1 ke 2 Cores, akan menghasilkan speedup sebanyak 1.6 kali.

ketika N semakin menuju infinity, speedup akan mendekati angka 1/S.

$$speedup \leq \frac{1}{S + \frac{(1-S)}{N}}$$

Hal ini berarti, bagian serial pada aplikasi tidak akan sebanding dengan peningkatan performa yang dihasilkan dari penambahan core.

Simplenya, Semakin kita menambahkan banyak core, bagian pada aplikasi yang berjalan secara serial akan membatasi speedup.

5. User Threads and Kernel Threads

1. User Threads → Management nya dilakukan oleh user-level thread library. 3 Thread Library Utama :
    a. POSIX Pthreads

b. Windows Threads

c. Java Threads

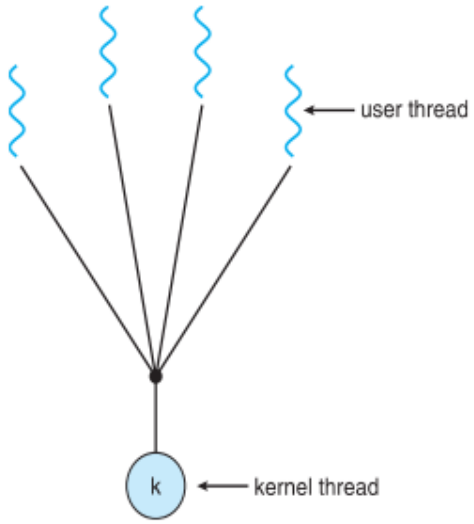
2. Kernel Thread → disediakan oleh kernel.

Contohnya : Hampir semua OS general purpose, termasuk Windows, Solaris, Linux, Tru64 UNIX, Mac OS x.

6. Multithreading Models

a. Many To One

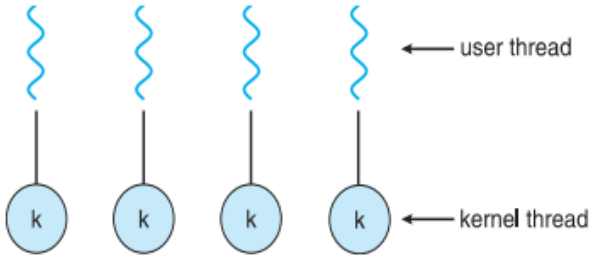
Banyak thread-thread user-level dipetakan / dihubungkan pada 1 kernel thread. Jika 1 thread terblokir, maka semua thread lainnya akan ikut terblokir. Beberapa thread bisa saja tidak dapat berjalan dalam paralel pada MultiCore System, karena hanya 1 User-Thread yang bisa masuk dalam Kernel Thread pada 1 waktu. Sedikit sekali sistem pada masa kini yang menggunakan model ini yaitu Solaris Green Threads dan GNU Portable Threads.



b. One To One

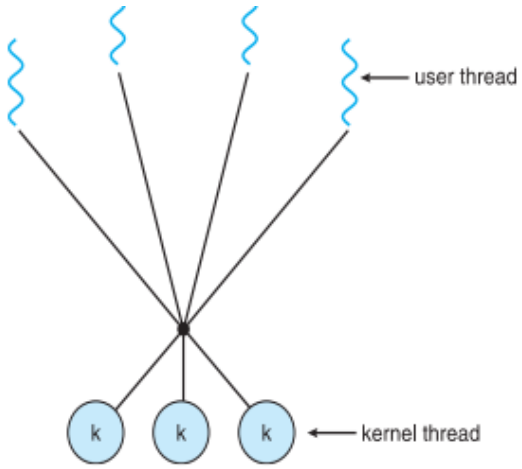
Setiap User-Level Thread memiliki Kernel Threadnya masing-masing. Ketika membuat User-Level Thread, maka akan membuat sebuah Kernel-Thread. Lebih Menggunakan Concurrency (Serempak) dibanding Many To One. Jumlah thread

per Process dapat terbatasi ketika beban kerja berlebih. Contohnya : Windows, Linux, dan Solaris9.



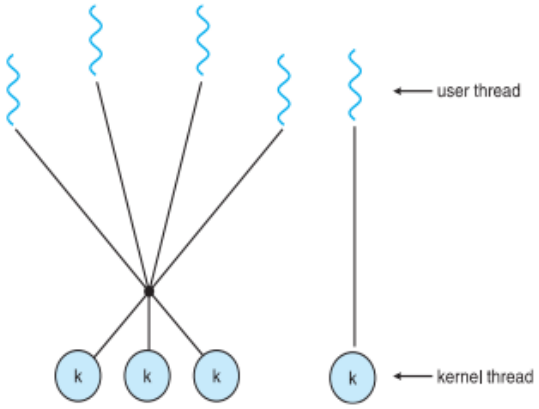
c. Many To Many Model

Beberapa User-Level Threads dapat dipetakan / dihubungkan ke lebih dari 1 Kernel Thread. Hal ini Mengizinkan OS untuk membuat jumlah Kernel Thread yang cukup. Contoh OS yang menggunakan ini : Solaris dengan versi dibawah 9 dan Windows ThreadFiber.



d. Two-Level Model

Mirip dengan Many To Many, tetapi memungkinkan untuk sebuah User Thread langsung terhubung dengan sebuah Kernel Thread. Contoh OS yang menggunakan Konsep ini : IRIX, HP-UX, TRU64 UNIX, dan Solaris 8.



B. PRACTICUM 1.1: Creating and Observing Threads

1. Process Vs Threads

a. Process

Process adalah program yang sedang dieksekusi oleh sistem Operasi. Dalam OS, process adalah identitas yang memiliki:

- Address Space yang terisolasi antara 1 process dengan yang lain
- Program Counter
- Register CPU
- Stack dan Heap
- File Descriptor
- Status Eksekusi (Ready, Running. Waiting, Terminated)

Secara formal, Process → {Code, Data, Heap, Stack, PCB}. Dimana:

- Code → Instruksi Program
- Data → Variable Global
- Heap → Alokasi dinamis
- Stack → Fungsi & Variabel Lokal

- PCB → Program Control Block (Metadata process di kernel)

Sifat Utama dari Process :

1. Isolasi Memori antar Process
2. Komunikasi antar Process menggunakan IPC (Inter-Process Communication) seperti Pipe, Socket, dan lain-lain
3. Context Switch relatif mahal / berat karena harus menyimpan dan memuat ulang seluruh state

b. Thread

Thread adalah unit eksekusi terkecil dalam sebuah process. Jika process adalah "Program yang berjalan", maka thread adalah alur eksekusi dalam process tersebut. Satu process bisa memiliki:

- Single Threaded (1 Thread)
- Multi-Threaded (Banyak Thread)

Secara teknis, thread memiliki sifat utama yaitu:

- Lebih ringan dibandingkan Process → karena thread adalah light-weight task
- Context Switch yang lebih cepat
- Komunikasi lebih mudah karena Shared Memory
- Memiliki risiko race condition jika tidak ada synchronization (Mutex, Semaphore)

2. Analogi Sederhana Mengenai Thread

Bayangin ges, Process itu adalah Rumah. Sedangkan, thread adalah orang didalam Rumah. Beberapa rumah (Process) terpisah satu sama lain. Orang-Orang (thread) dalam rumah yang sama berbatgi ruangan dan fasilitas.

3. Contoh Nyata : Web Browser

Misalnya satu tab = 1 Process. Dalam 1 tab ada beberapa thread seperti :

- Thread Rendering

- Thread Network
- Thread JavaScript

4. PROGRAM CODE : Prak8-1.c

```
// SAVW dengan nama prak8-1-1_NIM.c
// Mengaktifkan ekstensi GNU agar bisa menggunakan gettid()
#define _GNU_SOURCE
#include <sys/types.h>
#include <unistd.h>
#include <stdlib.h>
#include <stdio.h>
// API POSIX Thread
#include <pthread.h>

void* thread_function(void* arg) {
 printf("Child thread:\t pid=%d \t t_id (Linux)=%d \t t_id (P
osix)=%lu\n", getpid(), gettid(), pthread_self());
 while(1);
}

int main() {
    pthread_t thread1, thread2;
    int ret;

    printf("Main thread:\t pid=%d \t t_id (Linux)=%d \t t_id
(Posix)=%lu\n", getpid(), gettid(), pthread_self());
    // Pembuatan Thread Petama
    ret = pthread_create(&thread1, NULL, &thread_function, NU
LL);
    if(ret != 0) {
        perror("pthread_create: task1 \n");
        exit(EXIT_FAILURE);
    }
    // Pembuatan Thread Kedua
```

```
    ret = pthread_create(&thread2, NULL, &thread_function, NU
LL);
    if(ret != 0) {
        perror("pthread_create: task2\n");
        exit(EXIT_FAILURE);
    }

    while(1);

    return 0;
}
```

a. Penjelasan Teknis Program

Program ini membuat: 1 Main Thread & 2 Thread tambahan (thread1 dan thread2). Totalnya ada 1 Process dan 3 Thread.

1. Fungsi Penting : `pthread_create()`

```
pthread_create(&thread, NULL, function, arg);

/*
PARAMETERNYA :
1. Pointer ke variabel thread
2. Attribute (Biasanya / Defaultnya NULL)
3. Fungsi yang dijalankan thread
4. Argumen Fungsi
*/
```

NOTE :

- getpid() → Menghasilkan ID Process yang sama untuk semua thread
- gettid() → Thread ID yang unik untuk masing-masing thread
- pthread_self() → ID Thread versi POSIX dengan level user space

Hal ini menjadikan 1 thread memiliki 2 identitas yaitu sebagai POSIX ID dan Linux TID.

b. Menjalankan dan Observasi Program

1. Compile Program

```
gcc prak8-1.c -o prak8-1 -pthread
```

2. Run Program

```
./prak8-1
```

3. Untuk melihat berapa process mana yang berjalan : `top`
4. Untuk menampilkan thread yang sedang berjalan : `top -H`
5. Bisa juga sekaligus untuk melihat processes dan threads: `ps -eLo pid,tid,comm | grep prak8-1-1`

```
vboxuser@Ubuntu:~/Desktop/Week03$ ps -eLo pid,tid,comm | grep prak8-1
   3014    3014 prak8-1
   3014    3015 prak8-1
   3014    3016 prak8-1
```

c. 2 Cara Identifikasi Thread di Linux berdasarkan Lapisan

1. `pthread_self()` → Identitas Thread di Level POSIX (User Space)

`pthread_self()` adalah fungsi dari POSIX Threads. Karakteristiknya:

- Mengembalikan tipe `pthread_t`
- Dikelola oleh library pthread
- Bersifat portble (Berjalan di Linux, MacOS, dll)
- Biasanya Digunakan untuk
    - Sinkronisasi `pthread_equal`
    - Thread Management `pthread_join`
    - Identifikasi internal antar thread

2. `gettid()` → Identitas Thread di Level Kernel (Linux-Specific)

`gettid()` adalah SYSTEM CALL Linux, karakteristiknya :

- Mengembalikan Thread ID Kernel
- Unik secara global dalam sistem
- Digunakan oleh :
  - Scheduler kernel
  - CPU Tracking
  - `/proc`
  - `ps -L`
  - `top -H`
  - Tidak portable hanya khusus linux saja

```
vboxuser@Ubuntu:~/Desktop/Week03$ ./prak8-1
Main thread:     pid=3014        t_id (Linux)=3014       t_id (Posix)=123556333627200
Child thread:    pid=3014        t_id (Linux)=3015       t_id (Posix)=123556329420480
Child thread:    pid=3014        t_id (Linux)=3016       t_id (Posix)=123556321027776
```

C. PRACTICUM 1.2 : Using `pthread_join()`

1. Remove All while(1) Statement, dan apa yang terjadi ?

```
// SAVE dengan nama prak8-2-1_NIM.c
// Mengaktifkan ekstensi GNU agar bisa menggunakan gettid()
#define _GNU_SOURCE
#include <sys/types.h>
#include <unistd.h>
#include <stdlib.h>
#include <stdio.h>
// API POSIX Thread
#include <pthread.h>

void* thread_function(void* arg) {
 printf("Child thread:\t pid=%d \t t_id (Linux)=%d \t t_id (P
osix)=%lu\n", getpid(), gettid(), pthread_self());
```

```
 //while(1);
}

int main() {
    pthread_t thread1, thread2;
    int ret;

    printf("Main thread:\t pid=%d \t t_id (Linux)=%d \t t_id
(Posix)=%lu\n", getpid(), gettid(), pthread_self());

    // Pembuatan Thread Petama
    ret = pthread_create(&thread1, NULL, &thread_function, NU
LL);
    if(ret != 0) {
        perror("pthread_create: task1 \n");
        exit(EXIT_FAILURE);
    }
    // Pembuatan Thread Kedua
    ret = pthread_create(&thread2, NULL, &thread_function, NU
LL);
    if(ret != 0) {
        perror("pthread_create: task2\n");
        exit(EXIT_FAILURE);
    }

    return 0;
}
```

a. Jika `while(1)` dihapus dan tanpa menggunakan `pthread_join()` maka :

- Program bisa selesai sebelum thread selesai\
- Output akan berbeda-beda setiap Run
- Eksekusi tidak deterministik

Hal ini dapat dilihat ketika Infinite loop dihapus, maka :

- Main thread selesai sangat cepat
- Program exit sebelum thread1 dan thread2 sempat dieksekusi

Kalau Main thread selesai, maka :

- Process berakhir
- Semua thread langsung dihentikan oleh OS

<u>Program prak8-2.-1.c ⇒ Saya salah save nama hehehe</u>

```
vboxuser@Ubuntu:~/Desktop/Week03$ ./prak8-1-2
Main thread:     pid=3569        t_id (Linux)=3569       t_id (Posix)=126790938515264
Child thread:    pid=3569        t_id (Linux)=3570       t_id (Posix)=126790934722240
Child thread:    pid=3569        t_id (Linux)=3571       t_id (Posix)=126790926329536
vboxuser@Ubuntu:~/Desktop/Week03$ ./prak8-1-2
Main thread:     pid=3572        t_id (Linux)=3572       t_id (Posix)=126964411979584
vboxuser@Ubuntu:~/Desktop/Week03$ ./prak8-1-2
Main thread:     pid=3575        t_id (Linux)=3575       t_id (Posix)=134029016004416
vboxuser@Ubuntu:~/Desktop/Week03$ ./prak8-1-2
Main thread:     pid=3578        t_id (Linux)=3578       t_id (Posix)=124902655747904
vboxuser@Ubuntu:~/Desktop/Week03$ ./prak8-1-2
Main thread:     pid=3581        t_id (Linux)=3581       t_id (Posix)=137647962117952
vboxuser@Ubuntu:~/Desktop/Week03$ ./prak8-1-2
Main thread:     pid=3584        t_id (Linux)=3584       t_id (Posix)=129371616032576
vboxuser@Ubuntu:~/Desktop/Week03$ ./prak8-1-2
Main thread:     pid=3587        t_id (Linux)=3587       t_id (Posix)=123986712958784
Child thread:    pid=3587        t_id (Linux)=3588       t_id (Posix)=123986709051072
vboxuser@Ubuntu:~/Desktop/Week03$ ./prak8-1-2
Main thread:     pid=3590        t_id (Linux)=3590       t_id (Posix)=138503896422208
vboxuser@Ubuntu:~/Desktop/Week03$ ./prak8-1-2
Main thread:     pid=3593        t_id (Linux)=3593       t_id (Posix)=132784928859968
vboxuser@Ubuntu:~/Desktop/Week03$ ./prak8-1-2
Main thread:     pid=3596        t_id (Linux)=3596       t_id (Posix)=131681957750592
vboxuser@Ubuntu:~/Desktop/Week03$ ./prak8-1-2
Main thread:     pid=3599        t_id (Linux)=3599       t_id (Posix)=137255325271872
Child thread:    pid=3599        t_id (Linux)=3600       t_id (Posix)=137255322646208
vboxuser@Ubuntu:~/Desktop/Week03$ ./prak8-1-2
Main thread:     pid=3602        t_id (Linux)=3602       t_id (Posix)=126312099624768
vboxuser@Ubuntu:~/Desktop/Week03$
```

```
vboxuser@Ubuntu:~/Desktop/Week03$ ./prak8-1-2
Main thread:     pid=3431        t_id (Linux)=3431       t_id (Posix)=1355983028
16064
vboxuser@Ubuntu:~/Desktop/Week03$ ./prak8-1-2
Main thread:     pid=3434        t_id (Linux)=3434       t_id (Posix)=1241168077
31008
vboxuser@Ubuntu:~/Desktop/Week03$ ./prak8-1-2
Main thread:     pid=3437        t_id (Linux)=3437       t_id (Posix)=1399704295
11488
vboxuser@Ubuntu:~/Desktop/Week03$ ./prak8-1-2
Main thread:     pid=3442        t_id (Linux)=3442       t_id (Posix)=1363908204
68544
vboxuser@Ubuntu:~/Desktop/Week03$
```

1. **Jadi mengapa hanya Main Thread yang tampil ?**

Hal ini dikarenakan :

- Main thread langsung mencetak output
- Scheduler belum tentu sempat menjadwalkan thread1 dan thread2
- Program sudah terminate
- INGAT : KONSEP & MECHANISM OS YANG TERJADI :
  - Thread masuk ready Queue
  - Scheduler menentukan siapa yang jalan
  - Jika main selesai lebih dulu, maka :
    - Process dihentikan
    - Thread lain tidak sempat jalan

b. Modifikasi program menggunakan `pthread_join()` !

```
// SAVE dengan nama prak8-2-2_NIM.c
// Mengaktifkan ekstensi GNU agar bisa menggunakan gettid()
#define _GNU_SOURCE
#include <sys/types.h>
#include <unistd.h>
#include <stdlib.h>
#include <stdio.h>
```

```
// API POSIX Thread
#include <pthread.h>

void* thread_function(void* arg) {
 printf("Child thread:\t pid=%d \t t_id (Linux)=%d \t t_id (P
osix)=%lu\n", getpid(), gettid(), pthread_self());
 //while(1);
}

int main() {
    pthread_t thread1, thread2;
    int ret;

    printf("Main thread:\t pid=%d \t t_id (Linux)=%d \t t_id
(Posix)=%lu\n", getpid(), gettid(), pthread_self());

    // Pembuatan Thread Petama
    ret = pthread_create(&thread1, NULL, &thread_function, NU
LL);
    if(ret != 0) {
        perror("pthread_create: task1 \n");
        exit(EXIT_FAILURE);
    }
    // Pembuatan Thread Kedua
    ret = pthread_create(&thread2, NULL, &thread_function, NU
LL);
    if(ret != 0) {
        perror("pthread_create: task2\n");
        exit(EXIT_FAILURE);
    }

    // Menunggu kedua thread selesai
    pthread_join(thread1, NULL);
    pthread_join(thread2, NULL);
```

```
    return 0;
}
```

HASILNYA :

```
vboxuser@Ubuntu:~/Desktop/Week03$ ./prak8-2-2
Main thread:     pid=4878        t_id (Linux)=4878       t_id (Posix)=133180194158400
Child thread:    pid=4878        t_id (Linux)=4879       t_id (Posix)=133180191405760
Child thread:    pid=4878        t_id (Linux)=4880       t_id (Posix)=133180183013056
vboxuser@Ubuntu:~/Desktop/Week03$ ./prak8-2-2
Main thread:     pid=4918        t_id (Linux)=4918       t_id (Posix)=125240502101824
Child thread:    pid=4918        t_id (Linux)=4920       t_id (Posix)=125240491370176
Child thread:    pid=4918        t_id (Linux)=4919       t_id (Posix)=125240499762880
vboxuser@Ubuntu:~/Desktop/Week03$ ./prak8-2-2
Main thread:     pid=4921        t_id (Linux)=4921       t_id (Posix)=134905776588608
Child thread:    pid=4921        t_id (Linux)=4923       t_id (Posix)=134905765816000
Child thread:    pid=4921        t_id (Linux)=4922       t_id (Posix)=134905774208704
vboxuser@Ubuntu:~/Desktop/Week03$ ./prak8-2-2
Main thread:     pid=4924        t_id (Linux)=4924       t_id (Posix)=136173370840896
Child thread:    pid=4924        t_id (Linux)=4926       t_id (Posix)=136173359982272
Child thread:    pid=4924        t_id (Linux)=4925       t_id (Posix)=136173368374976
vboxuser@Ubuntu:~/Desktop/Week03$ ./prak8-2-2
Main thread:     pid=4927        t_id (Linux)=4927       t_id (Posix)=136646566516544
Child thread:    pid=4927        t_id (Linux)=4928       t_id (Posix)=136646563460800
Child thread:    pid=4927        t_id (Linux)=4929       t_id (Posix)=136646555068096
vboxuser@Ubuntu:~/Desktop/Week03$
```

C. Modifikasi Program agar Thread2 selalu berjalan setelah Thread1 selesai !

```
// SAVE dengan nama prak8-2-3_NIM.c
// Ini versi sequential => Minggu depan belajar Mutex dan Syn
chronization
#define _GNU_SOURCE
#include <sys/types.h>
#include <unistd.h>
#include <stdlib.h>
#include <stdio.h>
#include <pthread.h>

// Fungsi untuk thread1
void* thread1_function(void* arg) {
    printf("Thread 1 (Child 1):\n");
```

```
    printf("  pid=%d \t t_id (Linux)=%d \t t_id (Posix)=%lu\n
\n",
           getpid(), gettid(), pthread_self());
    return NULL;
}

// Fungsi untuk thread2
void* thread2_function(void* arg) {
    printf("Thread 2 (Child 2):\n");
    printf("  pid=%d \t t_id (Linux)=%d \t t_id (Posix)=%lu\n
\n",
           getpid(), gettid(), pthread_self());
    return NULL;
}

int main() {
    pthread_t thread1, thread2;
    int ret;

    printf("Main Thread:\n");
    printf("  pid=%d \t t_id (Linux)=%d \t t_id (Posix)=%lu\n
\n",
           getpid(), gettid(), pthread_self());

        // Buat Thread1
    ret = pthread_create(&thread1, NULL, thread1_function, NU
LL);
    if (ret != 0) {
        perror("pthread_create: thread1");
        exit(EXIT_FAILURE);
    }

    // Tunggu thread1 selesai
    pthread_join(thread1, NULL);

    // Buat Thread2
```

```
    ret = pthread_create(&thread2, NULL, thread2_function, NU
LL);
    if (ret != 0) {
        perror("pthread_create: thread2");
        exit(EXIT_FAILURE);
    }

    // Tunggu thread2 selesai
    pthread_join(thread2, NULL);

    printf("Main thread selesai.\n");

    return 0;
}
```

```
vboxuser@Ubuntu:~/Desktop/Week03$ ./prak8-2-3
Main Thread:
  pid=5079       t_id (Linux)=5079       t_id (Posix)=137244719114048

Thread 1 (Child 1):
  pid=5079       t_id (Linux)=5080       t_id (Posix)=137244715251392

Thread 2 (Child 2):
  pid=5079       t_id (Linux)=5081       t_id (Posix)=137244715251392

Main thread selesai.
vboxuser@Ubuntu:~/Desktop/Week03$ ./prak8-2-3
Main Thread:
  pid=5082       t_id (Linux)=5082       t_id (Posix)=132438124984128

Thread 1 (Child 1):
  pid=5082       t_id (Linux)=5083       t_id (Posix)=132438122559168

Thread 2 (Child 2):
  pid=5082       t_id (Linux)=5084       t_id (Posix)=132438122559168

Main thread selesai.
vboxuser@Ubuntu:~/Desktop/Week03$ ./prak8-2-3
Main Thread:
  pid=5085       t_id (Linux)=5085       t_id (Posix)=137929330661184

Thread 1 (Child 1):
  pid=5085       t_id (Linux)=5086       t_id (Posix)=137929328424640

Thread 2 (Child 2):
  pid=5085       t_id (Linux)=5087       t_id (Posix)=137929328424640
```

D. PRACTICUM 1.3 : Inter-Thread Communication

```
// SAVE dengan nama prak8-3-1_NIM.c

#include <stlib.h>
#include <unistd.h>
#include <pthread.h>
#include <stdio.h>

int g1 = 0;
```

```
void task1(int *counter) {
    while(*counter < 10) {
        printf("task1 count : %d\n", *counter);
        (*counter)++;
        sleep(1);
    }
    g1 = 10;
}

void task2(int *counter) {
    while(*counter < 10) {
        printf("task2 count : %d\n", *counter);
        (*counter)++;
        sleep(1);
    }
    g1 = 20;
}

int main() {
    pthread_t thrd1, thrd2;
    int a = 0, b = 0, ret;


    printf("Global variable g1 = %d\n", g1);

    ret = pthread_create(&thrd1, NULL, (void*)task1, (void*)&
a);
    if(ret != 0) {
        perror("pthread_create: task1\n");
        exit(EXIT_FAILURE);
    }

    ret = pthread_create(&thrd2, NULL, (void*)task2, (void*)&
b);
    if(ret != 0) {
        perror("pthread_create: task2\n");
```

```
        exit(EXIT_FAILURE);
    }

    pthread_join(thrd2, NULL);
    pthread_join(thrd1, NULL);

    printf("Global variabel g1 = %d\n", g1);

}
```

ALUR PROGRAM :

```
#include <stdio.h>
#include <unistd.h>
#include <pthread.h>
#include <stdio.h>

// Shared Global Variabel
int g1 = 0;

void task1(int *counter) {
    while(*counter < 10) {
        printf("task1 count : %d\n", *counter);
        (*counter)++;
        sleep(1);
    }
    g1 = 10;
}

/*
task1 :
- Loop 10 kali
- Cetak nilai Counter
- Increment
- Delay 1 detik
- Setelah selesai set g1 = 10
```

```
*/

void task2(int *counter) {
    while(*counter < 10) {
        printf("task2 count : %d\n", *counter);
        (*counter)++;
        sleep(1);
    }
    g1 = 20;
}

/*
task2 :
- Loop 10 kali
- Cetak nilai Counter
- Increment
- Delay 1 detik
- Setelah selesai set g1 = 20

*/

// STEP 1 : Main thread dimulai
int main() {
    pthread_t thrd1, thrd2;
    int a = 0, b = 0, ret;

    printf("Global variable g1 = %d\n", g1);

    // Membuat thread1, dan diberi alamat variabel a, dimana
a sekarang = 0
    ret = pthread_create(&thrd1, NULL, (void*)task1, (void*)&
a);
    if(ret != 0) {
        perror("pthread_create: task1\n");
        exit(EXIT_FAILURE);
    }
```

```
    ret = pthread_create(&thrd2, NULL, (void*)task2, (void*)&
b);
    if(ret != 0) {
        perror("pthread_create: task2\n");
        exit(EXIT_FAILURE);
    }

    // Kedua thread berjalan berbarengan
    // Interleaving Execution
    pthread_join(thrd2, NULL);
    pthread_join(thrd1, NULL);
    // Siapa yang selesai terakhir yang akan menentukan nilai
g1
    // misal task2 selesai duluan, maka nilai g1 adalah 20
    printf("Global variabel g1 = %d\n", g1);

}
```

HASILNYA KALAU SLEEP DIHILANGKAN → Bukti kalau race Condition

```
vboxuser@Ubuntu: ~/Desktop/Week03
task1 count : 9
task2 count : 6
task2 count : 7
task2 count : 8
task2 count : 9
Global variabel g1 = 20
vboxuser@Ubuntu:~/Desktop/Week03$ ./prak8-3-1
Global variable g1 = 0
task1 count : 0
task1 count : 1
task2 count : 0
task1 count : 2
task2 count : 1
task2 count : 2
task1 count : 3
task2 count : 3
task2 count : 4
task1 count : 4
task1 count : 5
task2 count : 5
task2 count : 6
task1 count : 6
task2 count : 7
task1 count : 7
task2 count : 8
task2 count : 9
task1 count : 8
task1 count : 9
Global variabel g1 = 10
```

1. Apakah Final Value dari Variabel g1 selalu sama setiap kali eksekusi ?

Tidak, Nilai variabel g1 tidak selalu sama setiap kali eksekusi. Hal ini terjadi karena Thread berjalan secara Concurrent dan bersama-sama mengakses dna menulis ke shared variabel g1. Hal ini dapat terjadi karena tidak adanya mekanisme sinkronisasi, sehingga nilai g1 dipengaruhi oleh thread terakhir yang selesai.

2. Modifikasi Program prak8-3-1.c sehingga thread1 dan thread2 berjalan secara concurrent, tetapi

sebelum program exit, nilai global g1 harus 20 dan main thread tidak boleh mengubah g1 !

Solusinya adalah :

- Menggunakan `pthread_mutex_t` → Melindungi akses shared variabel
- Menggunakan `pthread_cond_t` → Memberikan sinyal dari thread1 ke thread2
- Shared Flag → Menandakan thread1 sudah selesai

```
// Versi pakai Mutex
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <pthread.h>

int g1 = 0;

// Sinkronisasi
pthread_mutex_t mutex = PTHREAD_MUTEX_INITIALIZER;
pthread_cond_t cond = PTHREAD_COND_INITIALIZER;

int thread1_finished = 0;

// task1
void* task1(void* arg) {
    int *counter = (int*)arg;

    while(*counter < 10) {
        printf("task1 count : %d\n", *counter);
        (*counter)++;
        sleep(1);
    }

    pthread_mutex_lock(&mutex);
    g1 = 10;
    thread1_finished = 1;
```

```
    pthread_cond_signal(&cond);   // beri sinyal ke thread2
    pthread_mutex_unlock(&mutex);

    return NULL;
}

// task2
void* task2(void* arg) {
    int *counter = (int*)arg;

    while(*counter < 10) {
        printf("task2 count : %d\n", *counter);
        (*counter)++;
        sleep(1);
    }

    pthread_mutex_lock(&mutex);

    while(thread1_finished == 0) {
        pthread_cond_wait(&cond, &mutex);
    }

    g1 = 20;   // pasti akan terakhir

    pthread_mutex_unlock(&mutex);

    return NULL;
}

int main() {
    pthread_t thrd1, thrd2;
    int a = 0, b = 0;

    printf("Global variable g1 = %d\n", g1);

    pthread_create(&thrd1, NULL, task1, &a);
```

```
    pthread_create(&thrd2, NULL, task2, &b);

    pthread_join(thrd1, NULL);
    pthread_join(thrd2, NULL);

    printf("Global variable g1 = %d\n", g1);

    return 0;
}
```

```
// Versi pakai Ordering / Sequential
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <pthread.h>

int g1 = 0;

void* task1(void* arg) {
    int *counter = (int*)arg;

    while(*counter < 10) {
        printf("task1 count : %d\n", *counter);
        (*counter)++;
        sleep(1);
    }

    g1 = 10;
    printf("task1 set g1 = 10\n");
    return NULL;
}

void* task2(void* arg) {
    int *counter = (int*)arg;
```

```
    while(*counter < 10) {
        printf("task2 count : %d\n", *counter);
        (*counter)++;
        sleep(1);
    }

    g1 = 20;
    printf("task2 set g1 = 20\n");
    return NULL;
}

int main() {
    pthread_t thrd1, thrd2;
    int a = 0, b = 0;

    printf("Initial g1 = %d\n", g1);

    // Keduanya dibuat concurrent
    pthread_create(&thrd1, NULL, task1, &a);
    pthread_create(&thrd2, NULL, task2, &b);

    // Tunggu thread1 selesai dulu
    pthread_join(thrd1, NULL);

    // Lalu tunggu thread2 selesai
    pthread_join(thrd2, NULL);

    printf("Final g1 = %d\n", g1);

    return 0;
}
```

E. PRACTICUM 1.4 : Race Condition

```
// SAVE dengan nama prak8-4_NIM.c

// Program ini dibuat untuk mendemonstraksikan:
// Race Condition akibat operasi increment pada shared variab
el tanpa sinkronisasi
#include <unistd.h>
#include <stdlib.h>
#include <stdio.h>
#include <pthread.h>

#define NITERS 10000000

void count(void *arg);
unsigned int cnt = 0;

int main() {
    pthread_t tid1, tid2;

    // Kedua thread dibuat dan mengakses fungsi count masing-
masing
    pthread_create(&tid1, NULL, (void *)count, NULL);
    pthread_create(&tid2, NULL, (void *)count, NULL);

    /*
    Hal yang terjadi di CPU :
    - Load cnt dari memory
    - Tambah 1
    - Store kembali ke memory
    - Secara matematis harusnya x *  NITTERS, cnt = 20000000
    - Karena tidak adanya sinkronisasi, pasti akan menimpa sa
tu sama lain
    - Sehingga nilai NITTERS < 20000000
    */

    pthread_join(tid1, NULL);
```

```
    pthread_join(tid2, NULL);

    if(cnt != (unsigned)NITERS*2) {
        printf("Wrong... cnt=%d\n", cnt);
    } else {
        printf("OK...\tcnt=%d\n", cnt);
    }
    exit(0);
}

void count(void *arg) {
    int i;
    for(i = 0; i < NITERS; i++) {
        cnt++;
    }
}
```

```
vboxuser@Ubuntu:~/Desktop/Week03$ ./prak8-4
Wrong... cnt=10291325
vboxuser@Ubuntu:~/Desktop/Week03$ ./prak8-4
Wrong... cnt=14942844
vboxuser@Ubuntu:~/Desktop/Week03$ ./prak8-4
Wrong... cnt=11055140
vboxuser@Ubuntu:~/Desktop/Week03$ ./prak8-4
Wrong... cnt=10680939
vboxuser@Ubuntu:~/Desktop/Week03$ ./prak8-4
Wrong... cnt=14552126
vboxuser@Ubuntu:~/Desktop/Week03$ ./prak8-4
Wrong... cnt=11207278
vboxuser@Ubuntu:~/Desktop/Week03$ ./prak8-4
Wrong... cnt=10207510
vboxuser@Ubuntu:~/Desktop/Week03$ ./prak8-4
Wrong... cnt=10753947
vboxuser@Ubuntu:~/Desktop/Week03$ ./prak8-4
Wrong... cnt=10904175
```

1. Berapa Final Value cnt ?

2. Jelaskan Race Condition pada Program ini !